

Mini Project 5. Membandingkan FIR dan IIR pada Gangguan 50 Hz

Mini Project 5 melanjutkan langsung Mini Project 4. Mahasiswa tidak merekam ulang audio dan tidak mendesain filter optimal. Audio channel dari Bab 4 diberi hum 50 Hz secara terkontrol, lalu diproses dengan FIR high-pass 100 Hz dan IIR notch 50 Hz. Tujuan kegiatan adalah memahami perbedaan perilaku, selektivitas, fase, dan biaya komputasi. Hasil eksperimen menjadi input spesifikasi formal pada Mini Project 6.

Hasil Wajib Mini Project 5

- config/filter_experiment_config.json
- features/intermediate/mp05_filter_coefficients.npz
- data/processed/mp05_filter/nyala_001_channel_hum.wav
- data/processed/mp05_filter/nyala_001_fir_hp.wav
- data/processed/mp05_filter/nyala_001_iir_notch.wav
- results/metrics/mp05_filter_metrics.json
- results/figures/mp05_filter_response.png
- results/figures/mp05_filter_effect.png
- project_status.json dengan status ready_for_mp06

Prosedur Langkah demi Langkah

Langkah 1. Periksa Handoff Mini Project 4

Buka terminal pada folder audio_ai_embedded_project dan pastikan file berikut tersedia:

```
project_status.json
config/channel_config.json
data/processed/mp04_channel/nyala_001_channel.wav
features/intermediate/mp04_synthetic_rir.npz
results/metrics/mp04_channel_metrics.json
```

Buka project_status.json. Nilai status wajib ready_for_mp05. Jika tidak, selesaikan Mini Project 4 terlebih dahulu dan jangan membuat file pengganti secara manual.

Langkah 2. Buat Skrip Mini Project 5

Buat file mini_project_05.py di folder utama proyek. Skrip berikut memvalidasi handoff, menambahkan hum 50 Hz, menerapkan FIR dan IIR, menghitung respons filter serta metrik sederhana, lalu menyimpan artefak untuk Bab 6.

```
from pathlib import Path
import json
import time
import numpy as np
import matplotlib.pyplot as plt
from scipy.io import wavfile
from scipy.signal import firwin, iirnotch, freqz, lfilter, welch

ROOT = Path(__file__).resolve().parent
STATUS_PATH = ROOT / "project_status.json"
CHANNEL_CFG = ROOT / "config/channel_config.json"
MP04_METRICS = ROOT / "results/metrics/mp04_channel_metrics.json"
```

```

# 1. Validasi handoff Mini Project 4
with open(STATUS_PATH, "r", encoding="utf-8") as f:
    status = json.load(f)
if status.get("status") != "ready_for_mp05":
    raise RuntimeError("Mini Project 4 belum berstatus ready_for_mp05")

channel_audio = ROOT / status.get(
    "channel_audio",
    "data/processed/mp04_channel/nyala_001_channel.wav"
)
if not channel_audio.exists():
    raise FileNotFoundError("Audio channel Mini Project 4 tidak ditemukan")
if not CHANNEL_CFG.exists() or not MP04_METRICS.exists():
    raise FileNotFoundError("Konfigurasi atau metrik Mini Project 4 tidak lengkap")

# 2. Baca audio channel sebagai float -1..1
fs, raw = wavfile.read(channel_audio)
if raw.ndim > 1:
    raw = raw.mean(axis=1)
if np.issubdtype(raw.dtype, np.integer):
    scale = float(max(abs(np.iinfo(raw.dtype).min), np.iinfo(raw.dtype).max))
    x_channel = raw.astype(np.float32) / scale
else:
    x_channel = raw.astype(np.float32)

# 3. Tambahkan hum 50 Hz yang terkontrol
hum_frequency_hz = 50.0
hum_amplitude = 0.04
n = np.arange(len(x_channel))
hum = hum_amplitude * np.sin(2*np.pi*hum_frequency_hz*n/fs)
x_hum = x_channel + hum
# Sisakan headroom jika puncak terlalu besar
peak = np.max(np.abs(x_hum)) + 1e-12
if peak > 0.98:
    x_hum = x_hum * (0.98 / peak)

# 4. FIR high-pass sederhana: parameter tetap untuk MP5
fir_numtaps = 129
fir_cutoff_hz = 100.0
b_fir = firwin(
    fir_numtaps,
    fir_cutoff_hz,
    pass_zero=False,
    fs=fs
).astype(np.float64)
a_fir = np.array([1.0])
y_fir = lfilter(b_fir, a_fir, x_hum).astype(np.float32)
fir_group_delay_samples = (fir_numtaps - 1) // 2
fir_group_delay_ms = 1000.0 * fir_group_delay_samples / fs

# 5. IIR notch sederhana: parameter tetap untuk MP5
notch_q = 30.0
b_iir, a_iir = iirnotch(hum_frequency_hz, Q=notch_q, fs=fs)
y_iir = lfilter(b_iir, a_iir, y_fir).astype(np.float32)

# 6. Respons frekuensi kedua filter
f_resp, H_fir = freqz(b_fir, a_fir, worN=8192, fs=fs)
_, H_iir = freqz(b_iir, a_iir, worN=8192, fs=fs)
mag_fir_db = 20*np.log10(np.abs(H_fir) + 1e-12)
mag_iir_db = 20*np.log10(np.abs(H_iir) + 1e-12)

# 7. PSD input dan output
nperseg = min(2048, len(x_hum))
f_psd, P_in = welch(x_hum, fs=fs, nperseg=nperseg)
_, P_fir = welch(y_fir, fs=fs, nperseg=nperseg)
_, P_iir = welch(y_iir, fs=fs, nperseg=nperseg)

# Daya lokal di sekitar 50 Hz
def band_power(freq, psd, lo=45.0, hi=55.0):
    mask = (freq >= lo) & (freq <= hi)
    if np.count_nonzero(mask) < 1:
        return float("nan")
    integrate = getattr(np, "trapezoid", np.trapz)
    return float(integrate(psd[mask], freq[mask]))

p50_in = band_power(f_psd, P_in)
p50_fir = band_power(f_psd, P_fir)
p50_iir = band_power(f_psd, P_iir)

```

```

def attenuation_db(p_after, p_before):
    return float(10*np.log10((p_after + 1e-20)/(p_before + 1e-20)))

fir_50_db = attenuation_db(p50_fir, p50_in)
iir_50_db = attenuation_db(p50_iir, p50_in)

# Distorsi terhadap audio channel asli pada pita 150 Hz - 4 kHz.
# FIR dikompensasi delay hanya untuk metrik pembandingan ini.
Nspec = 8192
Xref = np.abs(np.fft.rfft(x_channel, n=Nspec))
Y_iir = np.abs(np.fft.rfft(y_iir, n=Nspec))
if fir_group_delay_samples < len(y_fir):
    y_fir_aligned = np.pad(
        y_fir[fir_group_delay_samples:],
        (0, fir_group_delay_samples)
    )[:len(x_channel)]
else:
    y_fir_aligned = y_fir
Y_fir = np.abs(np.fft.rfft(y_fir_aligned, n=Nspec))
f_spec = np.fft.rfftfreq(Nspec, 1/fs)
mask_voice = (f_spec >= 150) & (f_spec <= min(4000, fs/2))
ref_db = 20*np.log10(Xref/(Xref.max()+1e-12)+1e-9)
fir_db = 20*np.log10(Y_fir/(Y_fir.max()+1e-12)+1e-9)
iir_db = 20*np.log10(Y_iir/(Y_iir.max()+1e-12)+1e-9)
fir_dist = float(np.sqrt(np.mean((ref_db[mask_voice]-fir_db[mask_voice])**2)))
iir_dist = float(np.sqrt(np.mean((ref_db[mask_voice]-iir_db[mask_voice])**2)))

# 8. Estimasi waktu komputasi relatif di komputer (bukan benchmark MCU)
def bench(b, a, x, repeat=40):
    t0 = time.perf_counter()
    for _ in range(repeat):
        lfilter(b, a, x)
    return (time.perf_counter()-t0)/repeat

fir_time_s = bench(b_fir, a_fir, x_hum)
iir_time_s = bench(b_iir, a_iir, x_hum)

# 9. Folder output
out_audio = ROOT / "data/processed/mp05_filter"
out_feat = ROOT / "features/intermediate"
out_fig = ROOT / "results/figures"
out_met = ROOT / "results/metrics"
for p in [out_audio, out_feat, out_fig, out_met]:
    p.mkdir(parents=True, exist_ok=True)

# Simpan WAV PCM16
def save_pcm16(path, x):
    x = np.asarray(x, dtype=np.float32)
    peak = np.max(np.abs(x)) + 1e-12
    if peak > 0.999:
        x = x * (0.999/peak)
    wavfile.write(path, fs, np.int16(np.clip(x,-1,1)*32767))

save_pcm16(out_audio / "nyala_001_channel_hum.wav", x_hum)
save_pcm16(out_audio / "nyala_001_fir_hp.wav", y_fir)
save_pcm16(out_audio / "nyala_001_iir_notch.wav", y_iir)

np.savez_compressed(
    out_feat / "mp05_filter_coefficients.npz",
    b_fir=b_fir,
    a_fir=a_fir,
    b_iir=b_iir,
    a_iir=a_iir,
    frequency_hz=f_resp,
    H_fir=H_fir,
    H_iir=H_iir,
)

config = {
    "sample_rate_hz": int(fs),
    "hum_frequency_hz": hum_frequency_hz,
    "hum_amplitude": hum_amplitude,
    "fir": {
        "type": "highpass",
        "numtaps": fir_numtaps,
        "cutoff_hz": fir_cutoff_hz,
        "group_delay_samples": fir_group_delay_samples,
        "group_delay_ms": fir_group_delay_ms,
    }
}

```

```

    },
    "iir": {
        "type": "notch",
        "center_hz": hum_frequency_hz,
        "Q": notch_q,
    },
    "design_stage": "behavior_experiment_not_final_design",
    "recommended_problem_for_mp06": "suppress_50hz_hum_with_minimal_speech_distortion",
}
with open(ROOT / "config/filter_experiment_config.json", "w", encoding="utf-8") as f:
    json.dump(config, f, indent=2)

metrics = {
    "power_45_55hz_input": p50_in,
    "power_45_55hz_fir": p50_fir,
    "power_45_55hz_iir": p50_iir,
    "fir_attenuation_50hz_db": fir_50_db,
    "iir_attenuation_50hz_db": iir_50_db,
    "fir_spectral_distortion_150_4000hz_db_rms": fir_dist,
    "iir_spectral_distortion_150_4000hz_db_rms": iir_dist,
    "fir_group_delay_ms": fir_group_delay_ms,
    "fir_estimated_multiplies_per_sample": fir_numtaps,
    "iir_estimated_multiplies_per_sample": 5,
    "fir_pc_processing_time_s": fir_time_s,
    "iir_pc_processing_time_s": iir_time_s,
}
with open(out_met / "mp05_filter_metrics.json", "w", encoding="utf-8") as f:
    json.dump(metrics, f, indent=2)

# 10. Figure respons filter
fig, ax = plt.subplots(2, 1, figsize=(9, 7))
ax[0].plot(f_resp, mag_fir_db, label="FIR HP 100 Hz")
ax[0].plot(f_resp, mag_iir_db, label="IIR notch 50 Hz")
ax[0].set_xlim(0, min(800, fs/2))
ax[0].set_ylim(-80, 5)
ax[0].set_xlabel("Frequency (Hz)")
ax[0].set_ylabel("Magnitude (dB)")
ax[0].set_title("Magnitude response")
ax[0].legend(); ax[0].grid(alpha=0.2)
ax[1].plot(f_resp, np.unwrap(np.angle(H_fir)), label="FIR")
ax[1].plot(f_resp, np.unwrap(np.angle(H_iir)), label="IIR")
ax[1].set_xlim(0, min(800, fs/2))
ax[1].set_xlabel("Frequency (Hz)")
ax[1].set_ylabel("Phase (rad)")
ax[1].set_title("Phase response")
ax[1].legend(); ax[1].grid(alpha=0.2)
plt.tight_layout()
plt.savefig(out_fig / "mp05_filter_response.png", dpi=180)
plt.close()

# 11. Figure efek pada PSD
fig, ax = plt.subplots(2, 1, figsize=(9, 7))
show = min(len(x_hum), int(0.20*fs))
t = np.arange(show)/fs
ax[0].plot(t, x_hum[show], label="channel + hum", linewidth=0.8)
ax[0].plot(t, y_iir[show], label="IIR notch", linewidth=0.8, alpha=0.85)
ax[0].set_title("Waveform comparison")
ax[0].set_xlabel("Time (s)")
ax[0].legend(); ax[0].grid(alpha=0.2)
ax[1].plot(f_psd, 10*np.log10(P_in+1e-18), label="input")
ax[1].plot(f_psd, 10*np.log10(P_fir+1e-18), label="FIR")
ax[1].plot(f_psd, 10*np.log10(P_iir+1e-18), label="IIR")
ax[1].set_xlim(0, min(1000, fs/2))
ax[1].set_xlabel("Frequency (Hz)")
ax[1].set_ylabel("PSD (dB/Hz)")
ax[1].set_title("PSD before and after filtering")
ax[1].legend(); ax[1].grid(alpha=0.2)
plt.tight_layout()
plt.savefig(out_fig / "mp05_filter_effect.png", dpi=180)
plt.close()

# 12. Handoff ke Mini Project 6
status.update({
    "status": "ready_for_mp06",
    "last_completed_mini_project": 5,
    "filter_experiment_config": "config/filter_experiment_config.json",
    "filter_coefficients": "features/intermediate/mp05_filter_coefficients.npz",
    "filter_metrics": "results/metrics/mp05_filter_metrics.json",
    "filter_input_hum": "data/processed/mp05_filter/nyala_001_channel_hum.wav",
    "filter_output_fir": "data/processed/mp05_filter/nyala_001_fir_hp.wav",
})

```

```

"filter_output_iir": "data/processed/mp05_filter/nyala_001_iir_notch.wav",
"mp06_design_problem": "suppress_50hz_hum_with_minimal_speech_distortion",
})
with open(STATUS_PATH, "w", encoding="utf-8") as f:
    json.dump(status, f, indent=2)

print("Mini Project 5 selesai")
print("FIR attenuation 45-55 Hz (dB):", round(fir_50_db, 2))
print("IIR attenuation 45-55 Hz (dB):", round(iir_50_db, 2))
print("FIR spectral distortion RMS (dB):", round(fir_dist, 2))
print("IIR spectral distortion RMS (dB):", round(iir_dist, 2))
print("Status: ready_for_mp06")

```

Langkah 3. Jalankan Skrip

```
python mini_project_05.py
```

Skrip harus berakhir dengan pesan Mini Project 5 selesai dan status `ready_for_mp06`. Nilai attenuasi yang muncul dapat berbeda sedikit karena durasi audio dan estimasi PSD, tetapi IIR notch seharusnya menunjukkan penekanan yang sangat terlokalisasi di sekitar 50 Hz.

Langkah 4. Interpretasikan Dua Gambar

Buka `results/figures/mp05_filter_response.png`. Perhatikan bahwa FIR high-pass menekan seluruh wilayah frekuensi sangat rendah dan memiliki fase yang lebih teratur, sedangkan IIR notch memusatkan penolakan pada 50 Hz. Kemudian buka `mp05_filter_effect.png` dan pastikan puncak hum berkurang. Bandingkan apakah spektrum di atas 150 Hz tetap lebih dekat ke audio channel asli pada salah satu filter.

Langkah 5. Validasi Artefak dan Handoff ke Mini Project 6

Pastikan sembilan artefak wajib telah terbentuk dan buka `project_status.json`. Status akhir wajib `ready_for_mp06`. Jangan menghapus output FIR maupun IIR karena Bab 6 akan menggunakannya sebagai baseline sebelum desain berdasarkan spesifikasi.

Tabel 5.9. Kriteria lulus Mini Project 5 dan handoff ke Mini Project 6.

Artefak / pemeriksaan	Kriteria lulus	Digunakan pada MP6
<code>project_status.json</code>	<code>status = ready_for_mp06</code>	Mengaktifkan Mini Project 6
<code>filter_experiment_config.json</code>	Parameter FIR/IIR dan masalah desain tersimpan	Menjadi baseline spesifikasi
<code>mp05_filter_coefficients.npz</code>	Koefisien kedua filter tersedia	Perbandingan dengan desain Bab 6
<code>nyala_001_channel_hum.wav</code>	Audio dapat diputar, tidak clipping	Input desain formal
<code>nyala_001_fir_hp.wav</code>	Output FIR tersedia	Baseline FIR
<code>nyala_001_iir_notch.wav</code>	Output IIR tersedia	Baseline IIR
<code>mp05_filter_metrics.json</code>	Attenuation, distortion, delay dan biaya tercatat	Dasar keputusan desain
<code>mp05_filter_response.png</code>	Magnitude dan phase terbaca	Validasi perilaku
<code>mp05_filter_effect.png</code>	Puncak 50 Hz berkurang	Bukti efek filtering

Aturan serial

Jika waktu mendekati 30 menit, jangan mengubah cutoff, Q, atau jumlah tap untuk mencari hasil “terbaik”. Mini Project 5 adalah eksperimen perilaku dengan parameter tetap. Optimasi parameter dan metode desain dilakukan pada Mini Project 6.

Referensi Bab 5

1. Oppenheim, A. V., & Schaffer, R. W. Discrete-Time Signal Processing. Pearson.
2. Lyons, R. G. Understanding Digital Signal Processing. Pearson.
3. Zölzer, U. Digital Audio Signal Processing. Wiley.
4. Mitra, S. K. Digital Signal Processing: A Computer-Based Approach. McGraw-Hill.
5. Vaidyanathan, P. P. Multirate Systems and Filter Banks. Prentice Hall.
6. Proakis, J. G., & Manolakis, D. G. Digital Signal Processing: Principles, Algorithms, and Applications. Pearson.
7. ARM. CMSIS-DSP Software Library documentation.
8. Espressif Systems. ESP-DSP library documentation.